

MASTER PROMPT

HYPERLOCAL PRODUCT DISCOVERY PLATFORM

ADMIN CONTROL CENTER / ADMIN PANEL

COMPLETE FRONTEND IMPLEMENTATION 0→100%

You are a senior frontend architect, Next.js engineer, TypeScript engineer, React engineer, UI/UX designer, data-grid engineer, analytics engineer, security engineer, real-time systems engineer, API integration engineer, accessibility engineer, performance engineer, QA engineer, and production release engineer.

You are building ONLY the:

HYPERLOCAL ADMIN CONTROL CENTER

for the existing HyperLocal Product Discovery Platform.

This Admin Panel is for platform owners/operators/admin staff.

It is NOT the Customer App.

It is NOT the Shopkeeper App.

It is NOT a replacement for the FastAPI backend.

The Admin Panel is the control, monitoring, operations, moderation, analytics and management layer over the existing platform backend.

=====

0. PRIMARY OBJECTIVE

=====

Build a production-grade Admin Control Center where authorized administrators can:

VIEW MONITOR ANALYZE SEARCH FILTER INVESTIGATE MODERATE MANAGE CONFIGURE AUDIT CONTROL

the platform from one web application.

The Admin Panel must provide a unified view of:

- Customers
- Shopkeepers
- Businesses/Shops
- Products
- Product Variants
- Categories
- Brands
- Inventory
- Prices
- Price History
- Offers
- Search
- Search Quality
- Locations
- Notifications
- Imports
- POS
- Support
- Reviews/Reports if available
- Analytics
- Geography
- Platform Health
- Background Jobs
- Audit Logs
- Admin Users
- Permissions
- Feature Flags
- System Configuration

=====

1. BUSINESS PURPOSE

=====

Core platform:

Customer searches for a product.

System finds nearby shops/businesses.

Customer sees:

- product

- price
- availability
- distance
- rating where available
- shop information
- directions

Customer then visits the physical shop and purchases offline.

This is NOT a general e-commerce delivery system.

DO NOT introduce:

- grocery marketplace logic
- generic food delivery
- delivery rider management
- checkout
- cart
- shipping management

unless an explicitly approved future requirement adds them.

=====

2. ADMIN IS THE CONTROL CENTER

=====

The Admin Panel should follow:

SEE → UNDERSTAND → CONTROL → INVESTIGATE → CORRECT → MEASURE

Every major dashboard metric should support drill-down.

Example:

Stale Inventory = 4,230

Click

→ Stale Inventory List

→ Shop

→ Product

→ Inventory Record

→ Inventory History

→ Shopkeeper

The Admin Panel must not be a collection of disconnected charts.

=====

3. FRONTEND TECHNOLOGY CONTRACT

=====

Preferred stack:

Next.js TypeScript React Material UI MUI X Data Grid MUI X Charts where appropriate TanStack Query React Hook Form Zod WebSocket and/or SSE for real-time events Playwright Vitest / React Testing Library

Do not blindly install packages.

First inspect existing repository.

If equivalent dependencies already exist and are architecturally sound:

REUSE THEM.

=====

4. WHY THIS STACK

=====

Next.js:

- structured routing
- layouts
- production web application architecture
- server/client component model
- optimized web delivery

MUI:

- professional admin UI

- forms
- dialogs
- drawers
- tabs
- alerts
- navigation
- menus

MUI X Data Grid:

- large-data tables
- server-side filtering
- sorting
- pagination
- data loading
- row selection

MUI documents server-side Data Grid patterns specifically for growing datasets and supports server-side filtering, sorting and pagination.

TanStack Query:

- server-state fetching
- caching
- synchronization
- mutation
- query invalidation
- loading/error states

TanStack Query's documented model separates server state from general UI state and provides query/mutation mechanisms.

=====

5. ARCHITECTURAL PRINCIPLE

=====

Admin frontend architecture:

ADMIN UI ↓ VIEW / PAGE ↓ FEATURE STATE ↓ QUERY / MUTATION ↓ REPOSITORY / API CLIENT ↓ FASTAPI
ADMIN API ↓ AUTHORIZATION ↓ DATABASE / SERVICES

The Admin Panel must never directly connect to:

- PostgreSQL
- Redis

- S3
- AWS infrastructure

The browser communicates through approved backend APIs.

=====

6. SOURCE OF TRUTH

=====

Backend/database is authoritative for:

- users
- roles
- businesses
- products
- inventory
- prices
- offers
- verification
- notifications
- analytics
- system state

Frontend is only a presentation/control layer.

Do not create a second database in Next.js.

Do not persist authoritative platform data in browser storage.

=====

7. ADMIN AUTHENTICATION

=====

Admin authentication MUST be backend-controlled.

Do not assume Shopkeeper or Customer authentication can automatically be reused for Admin.

Inspect the existing backend admin-auth implementation.

Use whatever approved Admin authentication mechanism already exists.

If there is no implementation:

create only the frontend integration contract/placeholders and clearly report the backend requirement.

DO NOT invent a second admin authentication system inside Next.js.

=====

8. RBAC

=====

Admin authorization must be permission-aware.

Potential roles:

SUPER_ADMIN ADMIN VERIFICATION_ADMIN OPERATIONS_ADMIN SUPPORT_ADMIN CONTENT_ADMIN
ANALYST

Do not hardcode permissions into individual pages.

Use centralized permissions/capabilities.

Example:

customers.read customers.update customers.suspend

shops.read shops.approve shops.reject shops.suspend

products.read products.update products.merge

analytics.read

audit.read

settings.manage

=====

9. OWNER / SUPER ADMIN

=====

The platform owner should have a centralized administrator role.

Do not assume that every admin has all permissions.

Sensitive operations should be permission-protected.

=====

10. ADMIN SESSION

=====

App startup:

1. Initialize application
2. Check admin authentication
3. Load admin profile
4. Load permissions/capabilities
5. Load feature flags
6. Load essential platform state
7. Route to dashboard

Unauthenticated:

→ Admin Login

Authenticated:

→ Dashboard

Authenticated but forbidden:

→ 403 / Access Denied

Expired session:

→ re-authentication

=====

11. FINAL ROUTE STRUCTURE

=====

Use a centralized route structure.

Recommended:

/login

/dashboard

/customers /customers/[id]

/shopkeepers /shopkeepers/[id]

/businesses /businesses/[id]

/verification /verification/[id]

/products /products/[id]

/categories /categories/[id]

/brands /brands/[id]

/inventory /inventory/[id]

/pricing /pricing/[id]

/offers /offers/[id]

/search /search/quality /search/zero-results /search/trends

/locations /locations/map

/imports /imports/[id]

/pos /pos/[id]

/notifications /notifications/campaigns /notifications/[id]

/support /support/[id]

/analytics /analytics/customers /analytics/shopkeepers /analytics/products /analytics/shops /analytics/
search /analytics/geography

/audit

/system /system/health /system/jobs /system/flags /system/settings

/admin-users /admin-users/[id]

/settings

Use dynamic routes where entity details are required.

=====

12. APP SHELL

=====

Desktop-first administration interface.

Layout:

Logo Global Search Live Status Notifications Admin											
SIDEBAR MAIN CONTENT Dashboard Customers Shopkeepers											
Businesses Catalog Operations Discovery Engagement											
Analytics System											

Sidebar groups must be collapsible.

Remember selected navigation state.

=====

13. GLOBAL SEARCH

=====

Global admin search must support searching across multiple entity types.

Search:

Customer Shopkeeper Business Product Barcode Import Support Ticket Audit Event

Example:

Search:

8901030893456

Results:

Product Shops Inventory Price History

Another example:

Sharma Medical Store

Results:

Business Shopkeeper Products Inventory Verification Audit

=====

14. GLOBAL COMMAND CENTER

=====

Where appropriate support a command/search palette.

Potential actions:

Open Customer Open Shop Open Product Open Ticket Go to Verification Go to Imports Go to System Health

Do not make this unnecessarily complex.

=====

15. MAIN DASHBOARD

=====

Dashboard is the Admin operational overview.

TOP KPI CARDS:

Customers Shopkeepers Businesses Products Shop Products Inventory Records

SECONDARY KPIs:

Active Shops Pending Verification Stale Inventory Zero-Result Searches Imports Failed POS Sync Failed

ENGAGEMENT:

Searches Today Product Views Shop Views Directions Clicks Saved Products Saved Shops

LIVE:

Active Search Activity Recent Shop Updates Recent Inventory Updates Recent Registrations Recent Imports

=====

16. DASHBOARD DATE RANGE

=====

Support:

Today Yesterday 7 Days 30 Days 90 Days Custom Range

Do not hardcode date ranges in separate components.

Centralize analytics date filters.

=====

17. DASHBOARD FILTERS

=====

Potential filters:

City State Category Business Type Date Range Shop Status Customer Status

Only show filters supported by backend.

=====

18. LIVE DASHBOARD

=====

Dashboard may receive selected real-time events through:

WebSocket or SSE

Examples:

Customer search Shopkeeper registration Inventory update Price update Import completion Verification submission Support ticket

Do NOT stream every database query.

Only meaningful operational events should be realtime.

FastAPI supports WebSockets for persistent two-way connections; use realtime transport only where the product benefits from it.

=====

19. CUSTOMER MANAGEMENT

=====

CUSTOMERS:

- all customers
- active
- inactive
- suspended
- recently registered
- recently active

Table:

Name Phone/email as permitted Status Registered At Last Active Location summary where permitted Searches Saved Items Actions

=====

20. CUSTOMER DETAIL

=====

Customer detail page:

Overview Profile Account Status Authentication Status Registration Date Last Active

Activity:

Searches Viewed Products Viewed Shops Saved Products Saved Shops Notifications

Locations:

Saved Addresses Recent Location information where policy/backend allows

Support:

Tickets Reports

Audit:

relevant account audit events

=====

21. CUSTOMER ADMIN ACTIONS

=====

Depending on permission:

View Restrict Suspend Reactivate Review support issue

Sensitive actions require confirmation.

Never perform irreversible actions through an accidental click.

=====

22. CUSTOMER PRIVACY

=====

Do not expose unnecessary private data.

Use:

masking permission checks minimum necessary display

Never display:

passwords OTP authentication tokens security credentials

=====

23. SHOPKEEPER MANAGEMENT

=====

SHOPKEEPERS:

All New Active Suspended Incomplete Inactive

Columns:

Name Shop Category Status Verification Products Inventory Last Activity Created

=====

24. SHOPKEEPER DETAIL

=====

Show:

Identity Profile Shop Business Category Business Type Status Verification Location Product Count Inventory
Count Last Inventory Update Last Login/Activity where supported

Tabs:

Overview Business Products Inventory Prices Offers Imports POS Notifications Support Audit

=====

25. BUSINESS / SHOP MANAGEMENT

=====

Businesses table:

Shop Name Owner Category Location Status Verification Product Count Inventory Freshness Created Last
Updated

Actions:

View Edit if permitted Approve Reject Suspend Reactivate Archive

=====

26. BUSINESS DETAIL

=====

Show:

Business Information Location Category Business Type Contact Operating Hours Status Verification
Documents Products Inventory Pricing Offers History

=====

27. VERIFICATION CENTER

=====

Central verification queue:

Pending Under Review Approved Rejected Needs Correction

Filters:

Category Date Status Reviewer Location

=====

28. VERIFICATION DETAIL

=====

Show:

Shopkeeper Business Category Submitted information Documents Location Verification attempts Previous
decisions Reviewer Timeline

Actions:

Approve Reject Request Correction Put On Hold

Every decision:

Admin Time Reason Action

must be logged.

=====

29. PRODUCT CATALOG

=====

PRODUCT MANAGEMENT:

Products Variants Brands Categories Subcategories Identifiers Barcodes

Product table:

Name Brand Category Variant Identifier Status Shop Count Created Updated

=====

30. PRODUCT DETAIL

=====

Show:

Product Master Brand Category Subcategory Variants Identifiers Images Status

Related:

Shops Prices Availability Inventory History

=====

31. PRODUCT QUALITY CONTROL

=====

Identify:

Duplicate candidates Missing images Missing brand Missing category Invalid barcode Unmatched
identifiers Inactive products Products with no shop Products with suspicious data

=====

32. PRODUCT MERGE WORKFLOW

=====

If backend supports:

Candidate A

+ Candidate B

→ Compare

Show:

Name Brand Category Identifiers Variants Shop associations

Then:

Review → Confirm Merge

This is a HIGH-RISK operation.

Require:

confirmation reason audit

Do not silently merge.

=====

33. CATEGORY MANAGEMENT

=====

Current categories:

Pharmacy & Healthcare Beauty & Personal Care Furniture & Home Care Household Goods Sports, Fitness & Outdoor Books, Media & Stationery Automotive Parts & Tools Hardware Restaurants Transport Personal Transport / Personal Travel

Do NOT add:

Grocery General Food

unless a future product decision explicitly changes the catalog.

=====

34. CATEGORY CONFIGURATION

=====

If backend supports category configuration:

manage:

name description status sort order required fields optional fields feature capabilities

Do not implement category rules only in frontend.

=====

35. BRAND MANAGEMENT

=====

Support:

Brand list Search Status Product count Create/Edit if permitted Duplicate candidates

Use server-side search and pagination.

=====

36. INVENTORY CONTROL CENTER

=====

Inventory sections:

All Inventory In Stock Low Stock Out of Stock Unknown Stale Recently Updated Failed Updates

Table:

Shop Product Quantity where authorized Availability Source Freshness Last Updated

=====

37. INVENTORY FRESHNESS MONITORING

=====

Important platform metric:

Fresh Recent Stale Unknown

Dashboard:

Fresh % Recent % Stale % Unknown %

Click stale:

→ shops → products → last update → source

=====

38. INVENTORY SOURCES

=====

Show:

Manual Barcode Excel/CSV POS System

Do not invent sources not returned by backend.

=====

39. PRICE MANAGEMENT

=====

Show:

Current Price MRP if available Offer Price History

Price detail:

Shop Product Old Price New Price Changed By Changed At Source

=====

40. OFFER MANAGEMENT

=====

Sections:

Active Scheduled Expired Disabled Review

Actions:

View Disable if permitted Investigate Audit

=====

41. SEARCH CONTROL CENTER

=====

This is one of the highest-priority Admin modules.

SEARCH:

Search Volume Top Searches Zero Results Search Failures Latency Click-through Product matching Shop matching Trends

=====

42. ZERO-RESULT SEARCHES

=====

Show:

Query Count Date Range Location Potential category Matched products

Example:

"bosch drill 10mm"

184 searches 0 matching products

Admin actions:

Investigate Open Catalog Search Review Product Gap

Do not claim an exact product recommendation unless backend supplies it.

=====

43. SEARCH FUNNEL

=====

Track:

Search Started Search Submitted Results Returned Product Opened Shop Opened Directions Clicked Saved

Example:

10,000 searches ↓ 8,100 results ↓ 6,300 product views ↓ 4,200 shop views ↓ 1,800 directions

Charts must use real backend analytics only.

=====

44. SEARCH QUALITY

=====

Monitor:

Search success Zero-result rate Result count Click rate Product matching rate Shop matching rate Latency

If available:

exact matching fuzzy matching relevance score diagnostics

Do not implement search-engine logic inside the Admin frontend.

=====

45. GEOGRAPHIC CONTROL CENTER

=====

Use map visualization.

Show:

Customers Shops Search activity Product availability Category density Coverage

Filters:

City Category Date Shop Status

=====

46. COVERAGE ANALYSIS

=====

Admin should be able to inspect:

High customer demand Low shop coverage

Example:

Area A: Customers 12,000 Searches 20,000 Matching shops 32

Area B: Customers 8,000 Searches 14,000 Matching shops 3

This should be analytics, not invented business conclusions.

=====

47. SHOP / PRODUCT AVAILABILITY MAP

=====

Where backend supports:

select product → map nearby shops

Show:

shop marker availability price distance last update

Do not calculate conflicting business truth independently in browser.

=====

48. NOTIFICATION CENTER

=====

Admin notification management:

Templates Campaigns Scheduled Sent Failed Delivery status

Audience:

All Customers Selected Customers All Shopkeepers Selected Shopkeepers Category Location

Only expose targeting capabilities actually supported by backend.

=====

49. NOTIFICATION COMPOSER

=====

Fields:

Title Message Audience Schedule Deep Link Metadata

Preview:

Mobile notification preview

Actions:

Save Draft Schedule Send

Sending is HIGH IMPACT.

Require confirmation.

=====

50. NOTIFICATION SAFETY

=====

Never allow arbitrary unvalidated deep links.

Validate:

notification type entity ID supported route

=====

51. CONTENT / ANNOUNCEMENTS

=====

Admin may manage:

Home banners Announcements FAQs Help content Promotional cards System messages

Only implement if backend/content API exists.

=====

52. IMPORT CENTER

=====

Admin should monitor:

Excel CSV POS Bulk jobs

Statuses:

Uploaded Validating Processing Completed Partial Failed Cancelled

=====

53. IMPORT DETAIL

=====

Show:

Shop File Rows Valid Invalid Duplicates Processed Failed Started Finished Duration

Actions:

View Errors Download Report Retry when supported

=====

54. POS CONTROL CENTER

=====

Show:

connected disconnected syncing failed last sync products synced inventory synced

Do not couple the Admin UI to one vendor.

Use provider-neutral API models.

=====

55. SUPPORT CENTER

=====

Tickets:

Open In Progress Escalated Resolved Closed

Filter:

Customer Shopkeeper Business Product Search Technical Payment if future

=====

56. SUPPORT DETAIL

=====

Show:

Reporter Issue Type Description Attachments Related Customer Related Shop Related Product Status
Priority Assigned Admin Timeline

Actions:

Assign Change Status Add Internal Note Respond where API supports

=====

57. REVIEW / MODERATION

=====

If ratings/reviews/reporting exist:

Reviews Reported Flagged Hidden Moderation

Actions:

Review Keep Hide Escalate

Never permanently delete without backend support and proper permission.

=====

58. FRAUD / ABUSE MONITORING

=====

Future/available capabilities may include:

Suspicious accounts Spam Fake listings Repeated abuse Fake reviews Import abuse API abuse

Use:

flag review investigate

Do not automatically ban users based on frontend heuristics.

=====

59. ANALYTICS OVERVIEW

=====

Analytics should cover:

Customer Shopkeeper Business Product Inventory Search Geography Notifications Platform operations

=====

60. CUSTOMER ANALYTICS

=====

Metrics:

New Customers Active Users Returning Users Registrations Retention where available Searches Product Views Shop Views Directions Favorites

Date filters:

7D 30D 90D Custom

=====

61. SHOPKEEPER ANALYTICS

=====

Metrics:

New Shopkeepers Active Shopkeepers Active Businesses Product Additions Inventory Updates Price Updates Imports POS Sync Failed Imports

=====

62. PRODUCT ANALYTICS

=====

Metrics:

Top searched products Top viewed products Most available products Low coverage products Products with no shop Products with stale inventory

=====

63. BUSINESS ANALYTICS

=====

Metrics:

Active Shops New Shops Category distribution Location distribution Inventory freshness Product coverage

=====

64. SEARCH ANALYTICS

=====

Metrics:

Searches Successful searches Zero-result searches Product opens Shop opens Directions clicks

=====

65. NOTIFICATION ANALYTICS

=====

Where backend supports:

Sent Delivered Opened Clicked Failed

Do not fabricate delivery/open data.

=====

66. GEOGRAPHIC ANALYTICS

=====

Show:

Customer density Shop density Search density Category density Demand/coverage relation

Use aggregated backend data.

Do not download raw millions of coordinates unnecessarily.

=====

67. DATA VISUALIZATION RULE

=====

Use charts only where they add meaning.

Potential charts:

Line Bar Area Pie/donut Funnel Map Heatmap where backend supports

Do not make every number into a chart.

=====

68. TABLE RULE

=====

For large datasets:

USE SERVER-SIDE:

pagination sorting filtering search

Do not load thousands/millions of records into browser memory.

MUI X Data Grid explicitly supports server-side filtering, sorting and pagination for this pattern.

=====

69. DATA GRID RULE

=====

Reusable AdminDataGrid abstraction.

Capabilities:

columns server pagination server sorting server filtering search column visibility row selection bulk actions
refresh export if supported

Do not duplicate grid logic in every page.

=====

70. DATA GRID SELECTION

=====

Support where appropriate:

single selection multi-selection select page select all matching

Never accidentally operate on all records when user selected only visible records.

=====

71. BULK ACTIONS

=====

Examples:

Suspend Shops Approve Businesses Change Category Export Send Notification

For high-impact operations:

Require:

selection summary action reason confirmation

=====

72. EXPORT CENTER

=====

Exports may include:

Customers Shopkeepers Businesses Products Inventory Prices Search Analytics Audit Logs

Large exports should be asynchronous through backend jobs.

Do not generate enormous files entirely inside browser.

=====

73. AUDIT LOG CENTER

=====

Every sensitive admin action should be traceable.

Display:

Admin Action Entity Entity ID Timestamp Reason where applicable Result Request/Correlation ID where provided

Potential:

before after

when backend supplies it.

=====

74. AUDIT FILTERS

=====

Filter:

Admin Action Entity type Date Status Severity

Search:

Entity ID Admin Action

=====

75. ADMIN USER MANAGEMENT

=====

If backend supports:

Admin Users Roles Permissions Status Last Active

Actions:

Create Deactivate Change role Reset access where backend supports

High-impact changes require confirmation.

=====

76. FEATURE FLAGS

=====

Admin may view/manage feature flags if backend supports.

Examples:

BARCODE_SEARCH EXCEL_IMPORT POS OFFERS AI_SEARCH NEW_SEARCH_UI

Show:

Flag Status Environment Last Changed Changed By

Do not expose raw backend secrets.

=====

77. SYSTEM CONFIGURATION

=====

Potential controlled configuration:

Search radius Inventory freshness thresholds Import limits Notification limits Feature flags Result limits

Do NOT expose infrastructure secrets such as:

AWS secret keys database passwords Firebase Admin keys

=====

78. SYSTEM HEALTH

=====

Admin should be able to monitor:

API Database Redis/Valkey S3 Firebase Search Background jobs Notifications AWS/compute status where backend exposes it

Possible metrics:

status latency error rate last check incident

=====

79. BACKGROUND JOB CENTER

=====

Jobs:

Import POS sync Notification Export Analytics processing Image processing where applicable

Statuses:

Queued Running Completed Failed Cancelled

Admin actions:

View Retry Cancel where backend supports

=====

80. LIVE OPERATIONS

=====

Create a Live Operations page.

Show:

recent searches recent registrations inventory updates price changes imports verification activity support events

Use realtime events where available.

Do not poll every table aggressively.

=====

81. REALTIME ARCHITECTURE

=====

Initial page load:

REST API

Then:

WebSocket/SSE events

Flow:

Admin Page ↓ Initial Data API ↓ Realtime Connection ↓ Small Event Updates ↓ Query Invalidation / Local State Update

Do not rebuild the entire page for one event.

=====

82. TANSTACK QUERY RULE

=====

Use TanStack Query for server state.

Examples:

customersQuery shopsQuery productsQuery inventoryQuery analyticsQuery

Mutations:

approveBusinessMutation suspendShopMutation updateProductMutation

After mutation:

invalidate/update only affected queries.

TanStack Query documents queries, mutations and query invalidation as core patterns.

=====

83. CLIENT STATE

=====

Use lightweight client state only for:

sidebar theme dialogs temporary UI selected filters local drafts

Do not put all server data into a global client store.

=====

84. FORM ARCHITECTURE

=====

Use React Hook Form + Zod if not already using another sound solution.

Forms:

Create/Edit Business Product Category Brand Notification Admin User System Config

Validate client-side.

Backend remains authoritative.

=====

85. DIALOG ARCHITECTURE

=====

Reusable:

ConfirmationDialog DangerConfirmationDialog FormDialog FilterDrawer DetailDrawer InfoDialog

Do not create unique dialog implementations for every action.

=====

86. DRAWER VS PAGE

=====

Use drawer when:

quick detail small edit short review

Use full page when:

complex workflow large form deep investigation analytics

Maintain consistent navigation.

=====

87. ENTITY DETAIL ARCHITECTURE

=====

Use standardized detail layouts:

Header Status Summary Tabs Activity Audit

Example:

Business Detail → Overview → Verification → Products → Inventory → Pricing → Offers → History

=====

88. STATUS SYSTEM

=====

Centralize status display.

Examples:

ACTIVE INACTIVE PENDING VERIFIED REJECTED SUSPENDED STALE FAILED PROCESSING COMPLETED

Each status should have:

label semantic meaning display style optional icon

Do not duplicate status-color logic across files.

=====

89. DATE/TIME

=====

Use one date/time formatting utility.

Support:

absolute timestamps relative timestamps where appropriate

Example:

Updated 4 min ago

Click:

12 Sep 2026, 14:23:22

Do not calculate inconsistent times in multiple pages.

=====

90. MONEY

=====

Use safe monetary parsing/display.

Example:

₹248.00

Do not perform imprecise floating point business calculations in frontend.

Prefer backend-authoritative values.

=====

91. LOCATION / MAP

=====

Map functionality:

Shop locations Customer aggregate locations where permitted Coverage Search heatmaps where backend provides Selected business

Do not expose unnecessary individual customer location detail.

=====

92. PRIVACY BY DESIGN

=====

Admin power does NOT mean unrestricted raw data exposure.

Use:

Role-based visibility Field-level masking Audit Minimum necessary data

Never display:

OTP password Firebase private credentials AWS secrets database passwords

=====

93. ERROR HANDLING

=====

Centralized API error handling.

Map:

401 Unauthorized/session issue

403 Permission denied

404 Not found

409 Conflict

422 Validation

429 Rate limit

500+ Server error

Show user-friendly message.

=====

94. GLOBAL ERROR HANDLER

=====

Use reusable:

ErrorBoundary ApiErrorMessage ErrorState RetryButton

Do not let one API failure crash the entire dashboard.

=====

95. OFFLINE / NETWORK STATE

=====

Admin app should recognize:

Online Offline Reconnecting

When offline:

show status.

Do NOT allow false success.

Mutations should only report success after backend confirmation.

=====

96. SESSION EXPIRATION

=====

On 401:

attempt safe refresh/re-authentication according to backend contract.

If invalid:

clear protected state → Login

Avoid infinite retry loops.

=====

97. SECURITY

=====

Never store:

AWS secrets database credentials Firebase Admin keys private API keys

in frontend source.

Never put security-critical authorization only in UI.

Backend must remain authoritative.

=====

98. CSRF / COOKIE / TOKEN MODEL

=====

Inspect existing backend authentication architecture.

Use the backend's approved secure authentication mechanism.

Do NOT invent:

custom auth headers custom token storage custom admin session

without reviewing backend contract.

Do not store sensitive long-lived credentials in localStorage unless the backend/security architecture explicitly requires and accepts that design.

=====

99. API CLIENT

=====

ONE centralized API client.

Responsible for:

base URL authentication/session headers timeouts error mapping request IDs safe retries

Do not create:

ApiClientV2 AdminApiClientNew CustomerAdminHttp

unless there is a documented architectural reason.

=====

100. API MODULES

=====

Organize API access by domain.

Examples:

customersApi shopkeepersApi businessesApi productsApi inventoryApi pricingApi offersApi searchApi analyticsApi notificationsApi importsApi supportApi auditApi systemApi

Use typed responses.

=====

101. TYPESCRIPT

=====

Use strict TypeScript.

Avoid:

any

as much as possible.

Do not spread untyped API objects through the application.

Unknown API data must be safely parsed/validated.

=====

102. API MODEL VALIDATION

=====

Where appropriate validate external API responses.

Handle:

null missing field unknown enum unexpected shape

Gracefully show fallback states.

=====

103. PAGINATION

=====

For large tables:

server-side pagination.

Prefer cursor/keyset pagination if backend supports it.

Never fetch entire platform into browser.

=====

104. FILTERS

=====

Filters must be represented in URL/query state where practical.

Benefits:

refresh-safe shareable browser navigation reproducibility

Do not unnecessarily store all filter state only in memory.

=====

105. SORTING

=====

Server-side for large datasets.

Centralize sort definition.

Do not allow arbitrary unsafe sort field injection.

Use backend-supported field list.

=====

106. TABLE URL STATE

=====

Examples:

/customers?status=active&page=2

/products?category=hardware&sort=updated_at

/search/zero-results?range=30d

This improves operational usability.

=====

107. SAVED FILTERS

=====

Optional future/implemented capability:

Save Filter Load Filter Delete Filter

Useful for operational queues:

Pending Verification Stale Inventory Failed Imports

Only implement if it fits product scope.

=====

108. SEARCH INPUT DEBOUNCE

=====

Global and table searches should use debounce.

Do not call API on every keystroke.

Cancel obsolete requests where appropriate.

=====

109. RACE CONDITIONS

=====

If user performs:

Search A then Search B

Response B must not be overwritten by late response A.

Use query keys/request lifecycle correctly.

=====

110. LOADING STATES

=====

Every major page:

Loading

Every mutation:

Submitting... Saving... Approving... Suspending... Exporting...

Do not use endless full-screen loaders unnecessarily.

=====

111. EMPTY STATES

=====

Every table/list:

No customers No shops No products No pending verification No failed imports No notifications No tickets

Provide context-specific action.

=====

112. ERROR STATES

=====

Every major data section:

Error Retry

Do not show blank screens when API fails.

=====

113. PARTIAL FAILURE

=====

Dashboard may have:

Customers API success Inventory API failed

Do NOT fail entire dashboard.

Show successful sections.

Show inventory section error independently.

=====

114. REFRESH STRATEGY

=====

Support:

manual refresh automatic refresh where useful realtime update

Do not poll every page every few seconds.

=====

115. DATA CONSISTENCY AFTER ACTION

=====

Example:

Approve business → business list status updates → verification queue updates → dashboard counts update
→ activity/audit appears

Do not require full page reload after every mutation unless necessary.

=====

116. HIGH-RISK OPERATIONS

=====

Require confirmation for:

Suspend customer Suspend shop Merge products Bulk updates Delete/archive Disable offers Send broadcast Change system settings Change admin permissions

Confirmation should state:

WHAT WHO/WHAT IS AFFECTED WHY CONSEQUENCE

=====

117. REASON CAPTURE

=====

For sensitive actions:

Require reason.

Store reason through backend where supported.

Never rely only on browser audit.

=====

118. BULK OPERATION UX

=====

Workflow:

Select rows ↓ Review selected count ↓ Choose action ↓ Configure action ↓ Confirm ↓ Submit ↓ Show progress ↓ Result

Show:

successful failed skipped

=====

119. BACKGROUND JOB UX

=====

For long tasks:

Do not freeze browser.

Create backend job.

Frontend shows:

Queued Running Completed Failed

Allow user to continue other admin work.

=====

120. ANALYTICS PERFORMANCE

=====

Do not download raw event datasets into browser when aggregation can happen on backend.

Preferred:

FastAPI → aggregated analytics endpoint → TanStack Query → charts

=====

121. CHART INTERACTION

=====

Where useful:

Click chart point/bar → filtered table

Example:

Zero-result searches chart → selected date → zero-result list

Charts should support investigation.

=====

122. LIVE DATA + QUERY CACHE

=====

Realtime event:

inventory.updated

Frontend:

invalidate affected inventory query

OR

update specific cached record

Do not reload entire application.

=====

123. ADMIN NOTIFICATIONS

=====

Admin notification center should include:

System alerts Verification alerts Import failures POS failures High error-rate alerts Support escalation
Security alerts

Badge counts should come from backend/realtime source.

=====

124. INCIDENT / SYSTEM ALERTS

=====

System alert UI:

severity title message timestamp affected service status

Potential:

INFO WARNING CRITICAL

Never invent system health.

=====

125. AUDIT TIMELINE

=====

Entity detail pages should show activity timeline where backend supports:

created updated approved rejected suspended price changed inventory changed

=====

126. ADMIN UX

=====

Desktop:

optimized

Tablet:

usable

Mobile:

basic admin access where practical

Do not sacrifice desktop data-management usability for mobile styling.

=====

127. RESPONSIVE DESIGN

=====

Desktop widths:

comfortable tables side navigation multi-column dashboards

Small screens:

collapsible sidebar horizontal table scrolling stacked cards responsive drawers

=====

128. ACCESSIBILITY

=====

Support:

keyboard navigation focus states ARIA labels screen readers color contrast visible status text proper form labels

Do not use color alone to communicate important state.

=====

129. THEME

=====

Centralized Admin Design System.

Recommended visual direction:

deep blue electric blue white light gray green orange red for destructive errors

Use:

Material UI centralized theme consistent typography consistent spacing consistent border radius consistent elevation

Do not create random page-specific themes.

=====

130. DARK MODE

=====

Only implement if the existing design/product requires it.

If enabled:

centralize theme switching.

Do not create independent dark styles per page.

=====

131. ADMIN COMPONENT SYSTEM

=====

Reusable components:

AdminLayout Sidebar TopBar GlobalSearch PageHeader StatCard DataTable FilterBar FilterDrawer
DetailDrawer StatusBadge ConfirmationDialog DangerDialog EmptyState ErrorState LoadingState
ActivityTimeline MetricCard ChartCard MapCard AuditTimeline PermissionGuard FeatureGate

Do not create hundreds of unnecessary components.

=====

132. FEATURE ARCHITECTURE

=====

Recommended structure:

src/ app/ router/ providers/ theme/ layouts/

core/ api/ auth/ permissions/ realtime/ errors/ config/ utils/ components/

features/ dashboard/ customers/ shopkeepers/ businesses/ verification/ products/ categories/ brands/
inventory/ pricing/ offers/ search/ maps/ imports/ pos/ notifications/ content/ support/ analytics/ audit/
admin-users/ system/ settings/

main/ loading/ unauthorized/

Use existing architecture if it is already clean.

=====

133. FEATURE INTERNAL STRUCTURE

=====

For complex features:

feature/ api/ components/ hooks/ schemas/ types/ pages/ utils/

Do not blindly force layers into simple features.

=====

134. SINGLE RESPONSIBILITY

=====

Do not create giant files:

Dashboard.tsx AdminService.ts AppState.ts

that contain everything.

Split by domain.

=====

135. NO DUPLICATE API LOGIC

=====

If Customer screen and Admin screen use common API contracts:

keep shared typing/utilities where genuinely shared.

Do NOT create a dumping-ground shared folder.

=====

136. API VERSIONING

=====

Respect existing:

/api/v1

or existing backend versioning.

Do not hardcode API paths across every component.

Use centralized endpoint definitions.

=====

137. FEATURE FLAGS

=====

Frontend feature flags should come from centralized configuration/backend where available.

Examples:

enableRealtime enablePosAdmin enableAdvancedAnalytics enableReviewModeration

Do not scatter environment checks throughout pages.

=====

138. ADMIN PERMISSION GUARD

=====

Example:

```
<PermissionGuard permission="products.write">
```

Show Edit Product

```
</PermissionGuard>
```

But backend must still enforce permission.

=====

139. ROUTE GUARD

=====

Protected route:

Authentication

+ permission/capability

If unauthorized:

403 page

Never simply hide all errors.

=====

140. ACTION GUARD

=====

A button can be:

visible disabled hidden

depending on backend capability and UX.

But security always remains backend-side.

=====

141. GLOBAL ADMIN SEARCH RESULT TYPES

=====

Create a typed result union:

Customer Shopkeeper Business Product Barcode Ticket Import

Do not use an untyped generic result object.

=====

142. ROUTE DEEP LINK SAFETY

=====

Every dynamic entity route must handle:

entity exists entity deleted entity archived permission denied API failure

Fallback:

Not Found Access Denied Retry

=====

143. FILE EXPORT

=====

For large exports:

Request export job → backend → progress → download when ready

Do not block browser.

=====

144. FILE PREVIEW

=====

For supported documents:

show metadata preview if safe download where authorized

Do not expose private documents to unauthorized admins.

=====

145. IMAGE / MEDIA

=====

Product/shop images:

thumbnail preview broken image state loading

Do not load huge original assets unnecessarily.

=====

146. DOCUMENT SECURITY

=====

Verification documents:

private permission-controlled short-lived access where appropriate

Never expose storage credentials.

=====

147. ADMIN PROFILE

=====

Admin Profile:

Name Email Role Permissions summary Last login if backend provides

Actions:

Profile settings Security Logout

=====

148. ADMIN SETTINGS

=====

Settings:

Profile Notifications Appearance Timezone Language Session Security About

System settings remain under separate protected System area.

=====

149. TIMEZONE

=====

Use platform/backend timezone rules.

Analytics date filters must use explicit timezone context.

Avoid inconsistent day boundaries.

=====

150. LOGOUT

=====

Flow:

Admin Profile → Logout Confirmation → Secure logout → clear session → clear protected query/cache state
→ Login

Do not leave sensitive admin data accessible after logout.

=====

151. SECURITY CACHE CLEARING

=====

On logout:

clear sensitive client-side data.

Do not leave:

customer records shopkeeper records audit data

available through browser navigation after logout.

=====

152. TESTABLE ARCHITECTURE

=====

API logic must be testable independently.

Permissions must be testable.

Data mapping must be testable.

Filtering/formatting must be testable.

Do not tightly couple everything to React components.

=====

153. UNIT TESTS

=====

Test:

formatters validators permissions API mappers query key logic notification routing analytics transforms
table state status mapping

=====

154. COMPONENT TESTS

=====

Test:

login dashboard cards data tables filters dialogs permission guards empty states errors loading forms

=====

155. E2E TESTS

=====

Critical journeys:

Admin Login → Dashboard

Dashboard → Customer → Detail

Dashboard → Shopkeeper → Detail

Verification → Open Case → Approve/Reject

Products → Search → Open Detail

Inventory → Filter Stale → Open Shop

Search → Zero Results → Open Query

Notification → Compose → Confirm

Audit → Open Event

System → Health

=====

156. DATA GRID TESTS

=====

Test:

pagination sorting filters search row selection bulk selection empty loading server error

=====

157. PERMISSION TESTS

=====

Test:

authorized action unauthorized route unauthorized button read-only admin high-privilege admin

=====

158. REALTIME TESTS

=====

Test:

connect disconnect reconnect event received invalid event stale event query invalidation

=====

159. PERFORMANCE TESTS

=====

Test:

dashboard startup large tables rapid filtering rapid search chart rendering map rendering realtime events
memory usage

=====

160. BROWSER COMPATIBILITY

=====

Validate supported modern browsers.

Handle:

resize zoom keyboard slow machine network throttling

=====

161. ERROR RESILIENCE

=====

The Admin Panel must survive:

one failed API multiple failed APIs WebSocket disconnect expired auth invalid API response empty backend
data

without total application failure.

=====

162. DEPENDENCY RULE

=====

Before installing dependency:

inspect existing package.json.

Ask:

Can current package solve this?

If yes:

reuse.

If not:

add only if justified.

Avoid dependency sprawl.

=====

163. EXISTING CODE RULE

=====

This is an existing project.

FIRST:

inspect

THEN:

reuse refactor complete remove obsolete code only after proof

Do not rewrite the entire Admin project if parts already work.

=====

164. DELETE RULE

=====

Before deleting:

check imports routes API usage tests configuration deployment

Never delete based only on filename.

=====

165. API CONTRACT RULE

=====

Do not invent backend response formats when backend already exists.

Inspect FastAPI:

routes schemas status codes pagination filters permissions

Then map frontend.

=====

166. MISSING BACKEND FEATURE

=====

If a required Admin feature has no API:

DO NOT fake it.

Report:

MISSING BACKEND CONTRACT

Provide:

required endpoint required response shape required mutation required permission

But do not secretly implement a mock production workflow.

=====

167. NO FAKE ANALYTICS

=====

Never use fake:

customers shops revenue search counts ratings inventory charts

Mock data only for isolated UI development/testing.

Keep mocks separate.

=====

168. NO FAKE LIVE STREAM

=====

Do not simulate live activity in production.

Live data must come from backend realtime infrastructure.

=====

169. NO DIRECT DATABASE ACCESS

=====

Never:

Next.js → PostgreSQL

Never:

Browser → Redis

Never:

Browser → S3 private objects with permanent credentials

All through backend-approved interfaces.

=====

170. NO SECRET EXPOSURE

=====

Never put in frontend:

AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY DATABASE_URL Firebase Admin private key private service
account credentials provider secret keys

=====

171. ANALYTICS DATA PRINCIPLE

=====

Analytics must distinguish:

raw event aggregated metric derived metric

Frontend should consume backend-supported metrics.

Do not silently derive potentially misleading business metrics.

=====

172. AUDIT DATA PRINCIPLE

=====

Audit displayed by frontend is informational.

Backend must create authoritative audit records.

Never fabricate audit entries in browser storage.

=====

173. SYSTEM SETTINGS PRINCIPLE

=====

Admin UI should not expose raw infrastructure configuration.

Expose only safe application-level settings.

=====

174. HIGH-SCALE PRINCIPLE

=====

Design for growth.

Potential future scale:

100K+ customers 10K+ shopkeepers large product catalog large inventory large search history millions of analytics events

Frontend must:

paginate filter server-side aggregate analytics lazy-load avoid massive payloads

=====

175. INITIAL PAGE LOAD

=====

Do not load every module at startup.

Load:

authentication permissions dashboard essentials

Then lazy-load:

analytics large tables secondary modules

=====

176. CODE SPLITTING

=====

Use Next.js/React code-splitting patterns where appropriate.

Do not ship every heavy analytics/map module to every page.

=====

177. TABLE MEMORY

=====

Never store enormous row datasets unnecessarily.

Use server-side pagination.

Use virtualization through Data Grid capabilities.

=====

178. REACT RENDER PERFORMANCE

=====

Avoid unnecessary rerenders.

Use memoization only where profiling/structure justifies it.

Do not randomly add useMemo/useCallback everywhere.

=====

179. IMAGE OPTIMIZATION

=====

Use optimized image loading.

Do not load high-resolution originals into tables.

=====

180. MAP PERFORMANCE

=====

Use:

aggregation clustering server-side geographic aggregation

where necessary.

Do not render huge numbers of individual markers.

=====

181. URL STATE

=====

Keep operational filters/search in URL where appropriate.

This enables:

refresh back navigation bookmarking sharing within admin team

=====

182. BROWSER HISTORY

=====

Back button must behave naturally.

Example:

Customers → filtered list → customer detail → back

Return with:

same filters same page where possible

=====

183. DRAWER HISTORY

=====

If detail drawers are used, back/close behavior must be predictable.

=====

184. FORM DRAFTS

=====

Long forms should preserve drafts if user accidentally navigates away, where appropriate.

If unsaved changes:

show confirmation.

=====

185. UNSAVED CHANGES

=====

For important forms:

Save Cancel

If leaving with edits:

"Discard changes?"

Stay Discard

=====

186. NOTIFICATION COMPOSER SAFETY

=====

Before Send:

show:

Audience Count Message Schedule Deep Link

Require confirmation.

=====

187. BULK ACTION SAFETY

=====

Before bulk action:

show exact number:

"143 businesses selected"

Then:

action reason confirmation

=====

188. SYSTEM SETTINGS SAFETY

=====

Before changing system setting:

show:

Current Value New Value Impact Confirmation

Audit the change through backend.

=====

189. ADMIN PAGE DESIGN

=====

Each page should follow:

Page Header Description Primary Action Filters Content Pagination Status/Errors

Avoid clutter.

=====

190. DASHBOARD DESIGN

=====

Hierarchy:

1. critical issues
2. key metrics
3. trends
4. live operations
5. drill-down shortcuts

Do not make decorative charts more important than operational issues.

=====

191. SIDEBAR STRUCTURE

=====

Recommended:

DASHBOARD

PEOPLE Customers Shopkeepers

BUSINESSES Businesses Verification

CATALOG Products Categories Brands

OPERATIONS Inventory Pricing Offers Imports POS

DISCOVERY Search Search Quality Maps

ENGAGEMENT Notifications Content Support

ANALYTICS Overview Customers Shopkeepers Products Businesses Search Geography

=====

192. ADMIN DASHBOARD QUICK ACTIONS

=====

Quick actions:

Review Verification View Stale Inventory View Zero-result Searches View Failed Imports View Open Support Tickets View Critical Alerts

Actions should deep-link to exact filtered pages.

=====

193. NOTIFICATION BADGES

=====

Sidebar can show counts:

Verification 12 Imports 3 Support 7 Alerts 2

Counts must come from backend/realtime source.

=====

194. ADMIN HOME PERSONALIZATION

=====

Do not over-personalize.

Allow:

date range favorite pages saved filters later

=====

195. ADMIN ACCESSIBILITY

=====

Support:

keyboard-first operations focus management accessible dialogs accessible tables accessible charts where possible

=====

196. ADMIN SEARCH UX

=====

Global search:

type → debounce → result groups

Highlight matching terms where appropriate.

Keyboard:

Enter Arrow keys Esc

where practical.

=====

197. ADMIN SECURITY UX

=====

When permission denied:

show:

"You don't have permission to perform this action."

Do not reveal sensitive backend details.

=====

198. SYSTEM STATUS INDICATORS

=====

Top navigation may show:

API ● Database ● Realtime ●

Only if backend provides health data.

=====

199. FINAL ROUTE FLOW

=====

ADMIN LOGIN ↓ AUTH ↓ PERMISSIONS ↓ DASHBOARD ↓

																		Customers					
		Shopkeepers			Businesses			Verification			Products			Inventory			Pricing			Offers			
		Search			Maps			Imports			POS			Notifications			Support			Analytics			Audit
		System																					

=====

200. ADMIN DATA FLOW

=====

ADMIN BROWSER ↓ NEXT.JS ADMIN PANEL ↓ AUTH / PERMISSION ↓ FASTAPI ADMIN APIs ↓ BACKEND SERVICES ↓ POSTGRESQL / POSTGIS ↓ REDIS / CACHE ↓ S3 / STORAGE ↓ ANALYTICS / JOBS / SEARCH

Realtime:

BACKEND EVENT ↓ WEBSOCKET / SSE ↓ ADMIN PANEL ↓ QUERY INVALIDATION / STATE UPDATE

=====

201. CUSTOMER CONTROL FLOW

=====

ADMIN ↓ CUSTOMERS ↓ LIST ↓ FILTER ↓ DETAIL ↓ ACTIVITY ↓ SAVED / HISTORY ↓ SUPPORT ↓ AUDIT
↓ ACTION IF PERMITTED

=====

202. SHOPKEEPER CONTROL FLOW

=====

ADMIN ↓ SHOPKEEPERS ↓ SHOPKEEPER ↓ BUSINESS ↓ VERIFICATION ↓ PRODUCTS ↓ INVENTORY ↓
PRICES ↓ OFFERS ↓ IMPORTS ↓ POS ↓ ACTIVITY ↓ AUDIT

=====

203. PRODUCT CONTROL FLOW

=====

ADMIN ↓ PRODUCT ↓ MASTER ↓ VARIANTS ↓ BARCODES ↓ BRANDS ↓ CATEGORIES ↓ SHOPS ↓
INVENTORY ↓ PRICE ↓ HISTORY

=====

204. SEARCH CONTROL FLOW

=====

ADMIN ↓ SEARCH ↓ TOP SEARCHES ↓ ZERO RESULTS ↓ SEARCH QUALITY ↓ PRODUCT MATCHING ↓
SHOP MATCHING ↓ GEOGRAPHY ↓ TREND

=====

205. OPERATIONS CONTROL FLOW

=====

ADMIN ↓ OPERATIONS ↓ Inventory Pricing Offers Imports POS Notifications Support Jobs

=====

206. ANALYTICS FLOW

=====

BACKEND ↓ AGGREGATED ANALYTICS ↓ FASTAPI ↓ TANSTACK QUERY ↓ CHART/TABLE ↓ DRILL DOWN ↓
OPERATIONAL ACTION

=====

207. REALTIME FLOW

=====

BACKEND EVENT ↓ SSE / WEBSOCKET ↓ REALTIME SERVICE ↓ QUERY CACHE UPDATE / INVALIDATION ↓
ADMIN UI

=====

208. IMPLEMENTATION METHOD

=====

Do NOT implement everything in one giant uncontrolled change.

Use strict phases.

After every phase:

1. inspect result
2. run type checking
3. run tests

4. run build
5. verify routes
6. verify API contracts
7. verify permissions
8. fix problems
9. continue

=====

209. PHASE 1 — COMPLETE REPOSITORY AUDIT

=====

Inspect:

- Next.js version/config
- TypeScript
- package.json
- existing pages
- existing routes
- components
- state management
- API client
- authentication
- permissions
- current admin screens
- backend API contracts
- environment configuration
- build configuration
- tests

Deliver:

CURRENT STATE REPORT

Do not perform a giant rewrite.

=====

210. PHASE 2 — ARCHITECTURE CONTRACT

=====

Document:

routing layout permissions API layer query layer realtime layer component system theme error system

=====

211. PHASE 3 — FOUNDATION

=====

Implement/verify:

Next.js app TypeScript strictness theme layout sidebar topbar API client TanStack Query auth guard
permission guard error boundary global error handling

=====

212. PHASE 4 — AUTHENTICATION

=====

Implement:

Admin login integration session restore logout 401 handling 403 handling permission loading

Do not invent a separate auth backend.

=====

213. PHASE 5 — DASHBOARD

=====

Implement:

KPI cards alerts trends live events quick actions date filters

=====

214. PHASE 6 — CUSTOMERS

=====

Implement:

list search filter sort pagination detail activity actions

=====

215. PHASE 7 — SHOPKEEPERS

=====

Implement:

list search filter sort detail business relation activity actions

=====

216. PHASE 8 — BUSINESSES

=====

Implement:

list detail status location products inventory verification actions

=====

217. PHASE 9 — VERIFICATION

=====

Implement:

queue filters detail review approve reject correction request audit

=====

218. PHASE 10 — CATALOG

=====

Implement:

products variants brands categories identifiers quality control

=====

219. PHASE 11 — INVENTORY

=====

Implement:

all inventory freshness low stock out of stock history source

=====

220. PHASE 12 — PRICING / OFFERS

=====

Implement:

prices history offers status actions

=====

221. PHASE 13 — SEARCH

=====

Implement:

search metrics zero result trends quality drill-down

=====

222. PHASE 14 — IMPORT / POS

=====

Implement:

imports job status errors POS monitoring

=====

223. PHASE 15 — NOTIFICATIONS / CONTENT

=====

Implement:

notification center campaigns templates content where backend exists

=====

224. PHASE 16 — SUPPORT / MODERATION

=====

Implement:

tickets review/moderation where supported issue workflows

=====

225. PHASE 17 — ANALYTICS

=====

Implement:

customer analytics shopkeeper analytics product analytics business analytics search analytics geography

=====

226. PHASE 18 — AUDIT / ADMIN USERS

=====

Implement:

audit admin users roles permissions

=====

227. PHASE 19 — SYSTEM

=====

Implement:

live operations jobs health flags safe system settings

=====

228. PHASE 20 — REALTIME

=====

Implement:

connection reconnection event handling query invalidation live indicators

=====

229. PHASE 21 — PERFORMANCE

=====

Optimize:

Data Grid queries rendering bundle maps charts images realtime memory

=====

230. PHASE 22 — SECURITY

=====

Audit:

authentication authorization permissions PII exposure secret exposure XSS risks unsafe URLs unsafe
downloads route protection

=====

231. PHASE 23 — TESTING

=====

Run:

unit tests component tests integration tests E2E

=====

232. PHASE 24 — FINAL HARDENING

=====

Fix:

type errors build errors runtime errors broken routes API mismatches permissions loading empty error
responsive issues

=====

233. PHASE 25 — PRODUCTION RELEASE

=====

Validate:

production build environment variables API URL authentication permissions monitoring logging realtime performance

=====

234. GIT / CLINE DEVELOPMENT CONTRACT

=====

Cline MUST NOT work directly on:

main develop staging

Work only on:

feature/admin/ *bugfix/admin/* hotfix/admin/*

Workflow:

feature branch ↓ implementation ↓ tests ↓ analyze/typecheck ↓ commit ↓ push ↓ Pull Request ↓ CI ↓
develop ↓ staging ↓ final review ↓ main

Protected branches should require successful status checks before merge; GitHub documents required status checks for protected branches.

=====

235. CLINE WORK RULE

=====

Before every significant change:

1. Inspect
2. Understand
3. Find existing implementation
4. Reuse if possible
5. Plan minimum changes
6. Implement
7. Test
8. Verify
9. Report

Do not blindly generate duplicate architecture.

=====

236. FINAL SCREEN INVENTORY

=====

The implemented Admin Panel should cover the following logical areas:

AUTH

- Login
- Session
- Forbidden
- Unauthorized

DASHBOARD

- Overview
- Live Activity
- Alerts

CUSTOMERS

- List
- Detail
- Activity

SHOPKEEPERS

- List
- Detail
- Activity

BUSINESSES

- List
- Detail
- Location

VERIFICATION

- Queue
- Detail
- Review

CATALOG

- Products
- Variants
- Categories
- Brands
- Identifiers

INVENTORY

- Overview
- Freshness
- Low Stock
- History

PRICING

- Current Prices
- History

OFFERS

- Active
- Scheduled
- Expired

SEARCH

- Overview
- Top Searches
- Zero Results
- Quality
- Trends

MAPS

- Shop Map
- Coverage
- Search Geography

IMPORTS

- Jobs
- Details
- Errors

POS

- Status

- Sync
- History

NOTIFICATIONS

- Center
- Templates
- Campaigns
- Results

CONTENT

- Banners
- Announcements
- FAQs

SUPPORT

- Tickets
- Detail

ANALYTICS

- Overview
- Customers
- Shopkeepers
- Businesses
- Products
- Search
- Geography

AUDIT

- Audit Logs

SYSTEM

- Live Operations
- Background Jobs
- Health
- Feature Flags
- Admin Users
- Settings

=====

237. FINAL QUALITY RULE

=====

Do NOT declare Admin Panel complete because all routes exist.

Complete means:

☐ Authentication works ☐ Session works ☐ RBAC works ☐ Dashboard works ☐ Customers works ☐
Shopkeepers works ☐ Businesses works ☐ Verification works ☐ Products works ☐ Categories works ☐
Brands works ☐ Inventory works ☐ Pricing works ☐ Offers works ☐ Search analytics works ☐ Zero-result
analysis works ☐ Maps work where backend supports ☐ Imports work ☐ POS monitoring works where
backend supports ☐ Notifications work ☐ Support works ☐ Analytics works ☐ Audit works ☐ Admin users
work ☐ System health works where backend supports ☐ Feature flags work where backend supports ☐
Realtime works where backend supports ☐ Loading states work ☐ Empty states work ☐ Error states work
☐ Offline/reconnect states work ☐ Permission states work ☐ Large tables are server-side ☐ No fake
production data ☐ No secrets in frontend ☐ No direct database access ☐ No direct private S3 credentials ☐
No duplicate API clients ☐ No duplicate state architecture ☐ No dead routes ☐ No major accessibility issues
☐ No major responsive issues ☐ No obvious performance regressions ☐ TypeScript checks pass ☐ Tests
pass ☐ Production build passes

=====

238. FINAL REPORT REQUIRED

=====

After implementation provide:

A. CURRENT REPOSITORY AUDIT

B. FINAL ARCHITECTURE

C. FINAL FOLDER TREE

D. ROUTE MAP

E. ADMIN AUTH FLOW

F. RBAC/PERMISSION MAP

G. CUSTOMER MANAGEMENT FLOW

H. SHOPKEEPER MANAGEMENT FLOW

I. BUSINESS FLOW

J. VERIFICATION FLOW

K. PRODUCT/CATALOG FLOW

L. INVENTORY FLOW

M. PRICING/OFFER FLOW

N. SEARCH/QUALITY FLOW

O. MAP/GEO FLOW

P. IMPORT/POS FLOW

Q. NOTIFICATION FLOW

R. SUPPORT FLOW

S. ANALYTICS FLOW

T. REALTIME FLOW

U. AUDIT FLOW

V. SYSTEM CONTROL FLOW

W. API CONTRACTS USED

X. COMPONENTS CREATED

Y. QUERIES/MUTATIONS CREATED

Z. FILES CREATED

AA. FILES MODIFIED

AB. FILES REMOVED

AC. DUPLICATES REMOVED

AD. TESTS EXECUTED

AE. BUILD RESULT

AF. SECURITY CHECK

AG. PERFORMANCE CHECK

AH. MISSING BACKEND FEATURES

AI. MANUAL CONFIGURATION REQUIRED

AJ. REMAINING TODOs

=====

239. ABSOLUTE DO-NOT RULES

=====

DO NOT:

- rewrite working code unnecessarily
- create duplicate API clients
- create duplicate repositories/hooks
- create duplicate authentication
- create fake analytics
- create fake realtime data
- create fake customers
- create fake shops
- create fake inventory
- create fake search counts
- expose secrets
- connect directly to database
- connect directly to Redis
- expose private S3 credentials
- bypass backend permissions
- hardcode authorization
- load massive datasets into browser
- create giant uncontrolled global state
- create giant monolithic dashboard component
- create dead-end pages
- silently swallow API errors
- report failed mutation as success
- make high-risk actions one-click
- add Grocery/general Food category
- turn the platform into e-commerce/delivery through the Admin Panel

- add unnecessary infrastructure dependencies

=====

240. FINAL ADMIN PRINCIPLE

=====

This Admin Panel must become the operational brain of the platform.

The final experience should be:

SEE → SEARCH → FILTER → INVESTIGATE → CONTROL → AUDIT → ANALYZE → ACT

The administrator should be able to move naturally:

Dashboard → issue → entity → details → history → action → confirmation → result → audit

Everything must remain connected.

Every major metric must be actionable.

Every important operation must be traceable.

Every high-impact action must be permission-controlled.

Every large dataset must be server-side.

Every API failure must have a usable UI state.

Every sensitive data field must be appropriately protected.

Every realtime feature must degrade gracefully when disconnected.

The Admin Panel is a CONTROL CENTER, not merely a dashboard.

END OF MASTER PROMPT